DIGDIR/DIGDIR-AGENTS

Agents with their own computer

The security view

Security team · Security champions

THREAT MODEL

Assume the agent is hostile

- An agent runs code **nobody reviewed**: its own, dependencies, scripts from the web
- **Prompt injection** from issues, pages and packages can steer a well-meaning agent
- So we treat it as an attacker with **root inside its VM** and outbound network access
- Without a sandbox, it **runs as you**: your tokens, SSH keys and network

What we protect

assets

The host
your machine or the Kubernetes node, its files and its network

Secret values
GitHub tokens, model logins, API keys

Other agents and jobs
each with its own VM, disk and network

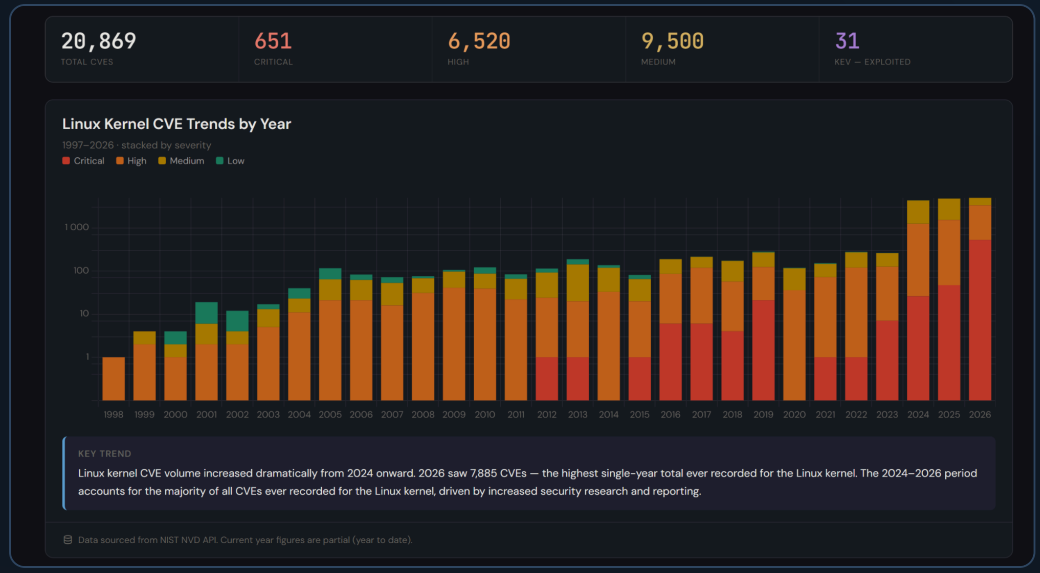
Where data goes
which hosts the agent may reach, and with which secrets

PART 1

The landscape

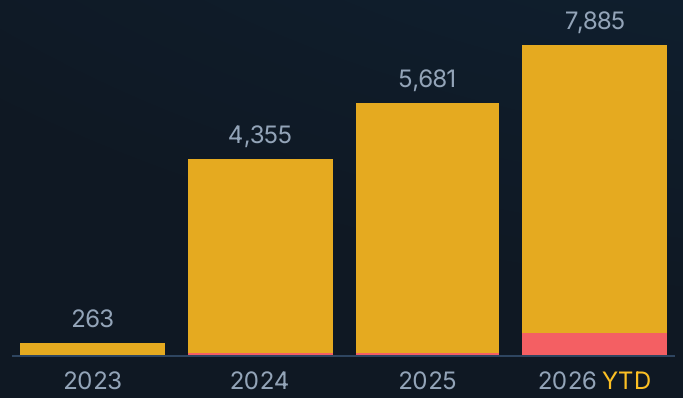
LINUX KERNEL · CVES PER YEAR

The kernel CVE flood



linuxcvetracker.com, NVD data, 21 Oct 2026. Log scale.

The same years on a linear scale



■ Critical: 26 → 47 → 529

Read it carefully

Since February 2024 the kernel is its own CVE authority and assigns a CVE to almost every stable fix. Most of the jump is **counting**, not new bugs. The critical count is the signal.

THE REGISTER · 22 JULY 2026

"Not feasible to prioritize individual kernel changes"

432

CVEs in two days

Published by the kernel CVE team over one Sunday and Monday in July.

AI

Bug hunting at scale

Linus Torvalds, May 2026: the security list has become "almost entirely unmanageable" because of AI-assisted bug hunting.



Patch the fleet, not the CVE

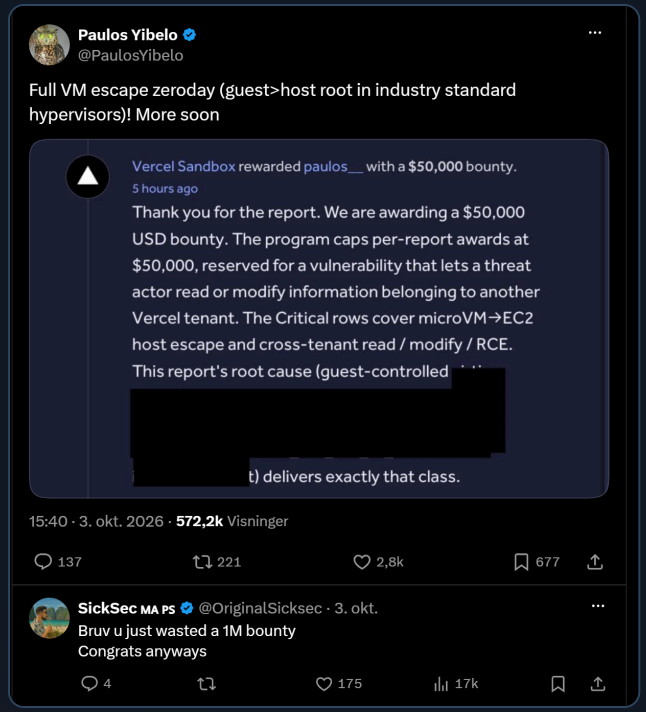
Jan Schaumann, Akamai: "This onslaught really shows it's not feasible to attempt to prioritize individual kernel changes."

A container shares the host kernel, so **every one of these** is a potential container escape.

Triage cannot keep up. The architecture has to limit how much host kernel untrusted code can reach.

CVE-2026-99999 · PWNESCAPE

Hypervisors aren’t magic either



- Found by Paulos Yibelo (pwn.ai) in **Vercel’s $1M Sandbox challenge**: the only Critical, on day three
- AI-assisted: **under $700** of API tokens
- The bug is in the **host kernel’s network stack**, not in KVM or Firecracker
- The guest sets **checksum metadata** in the virtio-net header. The host kernel trusts it and rewrites an IPv4 header it already validated
- Host memory read on Vercel’s production; host root in a matching KVM + Firecracker lab
- CVSS 9.3. Fixed upstream in `6785011f8b16`, stable kernels on 15 October. **No workaround:** patch the host and reboot

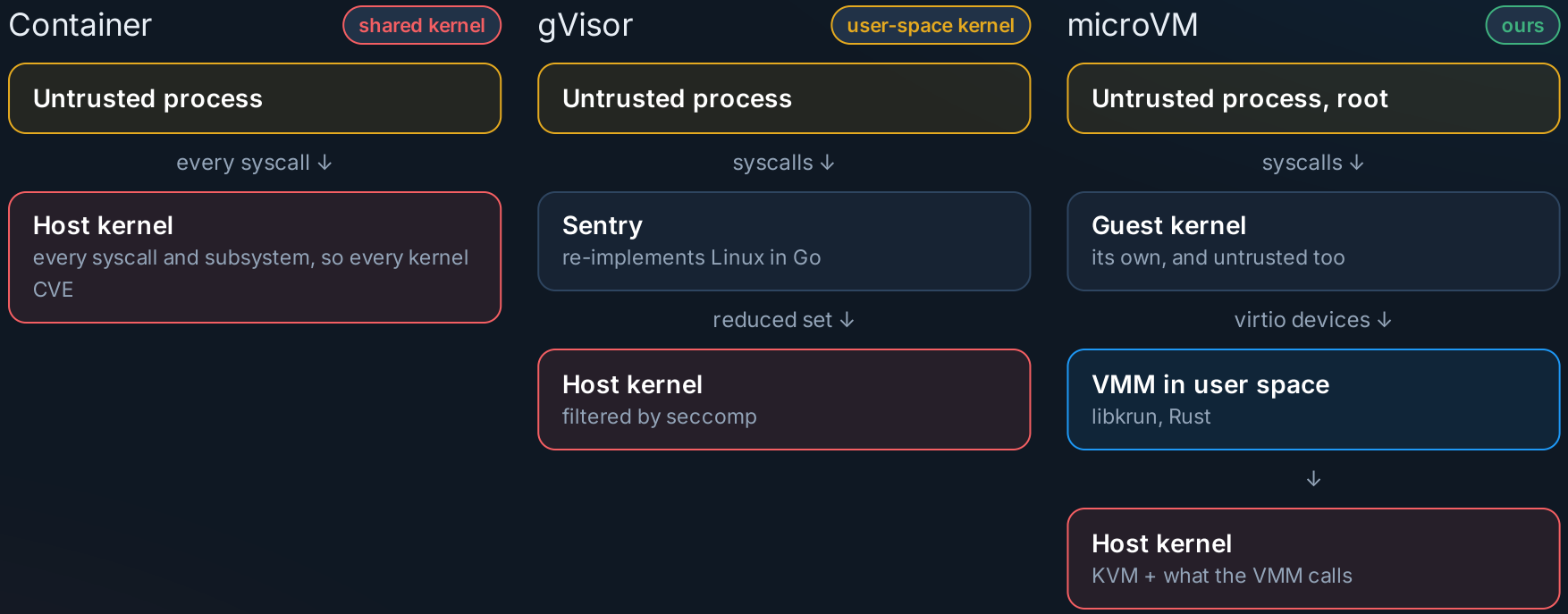
A VM boundary is only as narrow as the host code the guest can reach.

PART 2

Our boundary

ISOLATION

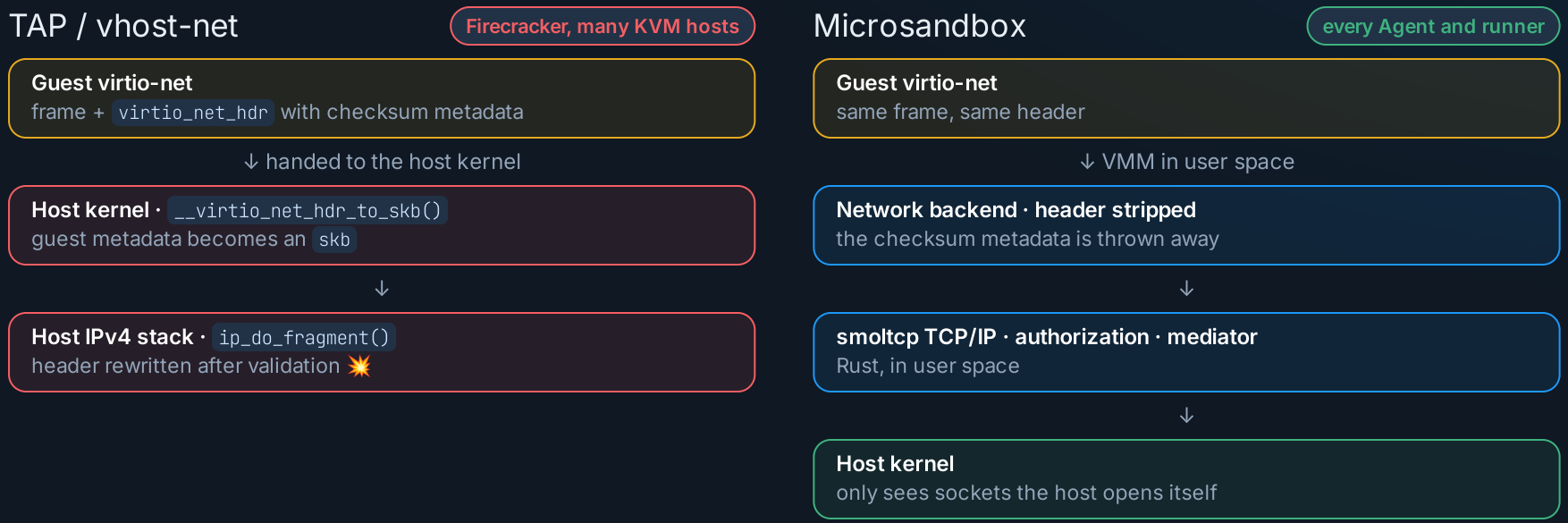
Where a guest's actions land



The question is never "VM or container?" but **which host code does guest-controlled data reach?**

CASE STUDY · CVE-2026-99999

The PwnEscape path ends in user space for us

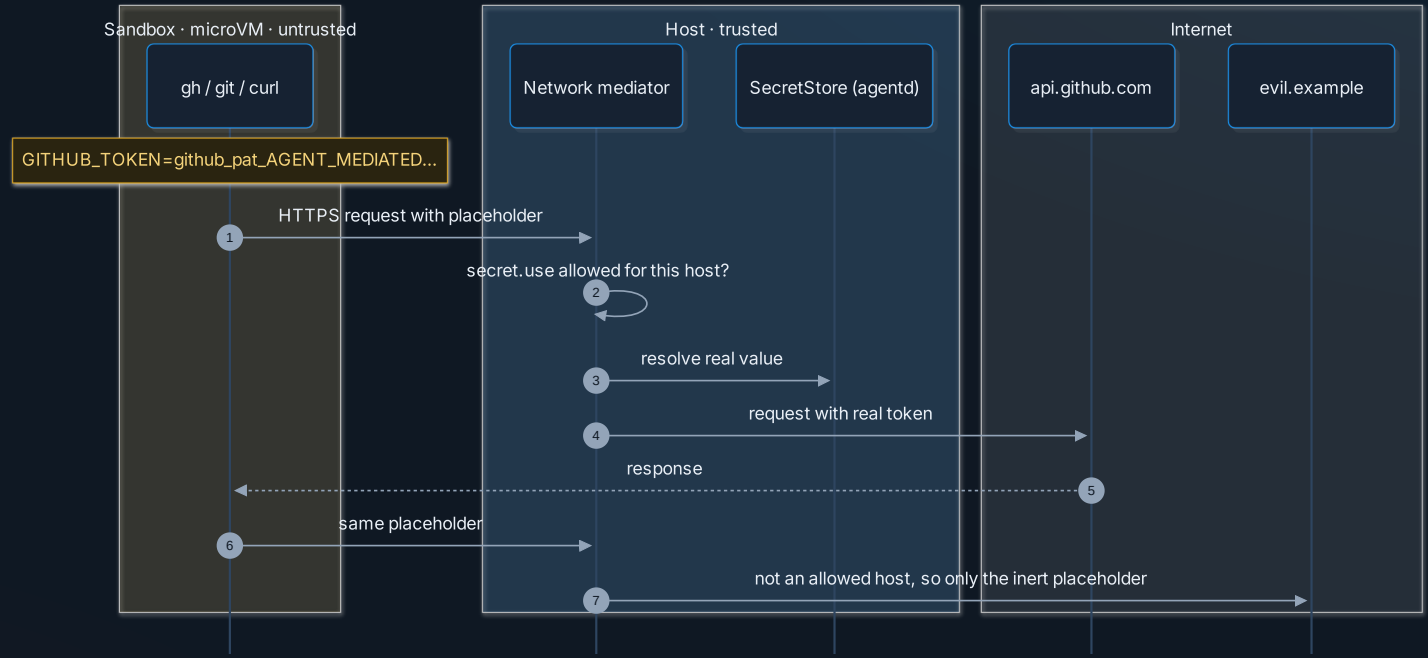


We did not predict this bug. The design keeps guest-controlled bytes away from the host kernel, so a whole class of bugs can't be reached.

AWS: "self-managed KVM hosts using virtio-net/TAP networking can be affected." Microsandbox never selects libkrun's TAP backend.

SECRETS AND EGRESS

Secrets never enter the Sandbox



Every layer ends on the host · `sandbox-authorization` for every check · Per-secret `allowedHosts` · Fails closed

AGENTCTL/CHANGELOG.MD · SECURITY

Found and fixed

Bypass after an agentd restart

Changing a running Agent's resources after a restart gave it network access around authorization and secret mediation.

Tunnelling through an allowed host

Non-HTTP traffic inside an HTTPS connection to an allowed host bypassed authorization. WebSockets still work.

`.env` in bind mounts

Applying an Agent rejects bind mounts that contain `.env` files, case-insensitively and regardless of ignore rules.

TLS dependencies

rustls advisories assessed and upgraded. Certificates chaining through the distrusted emSign intermediates are rejected.

The boundary is treated as a **security boundary**: fixes are released, and the changelog's Security section names them.

CI runs `cargo deny` for advisories and dependency sources on every change.

HONEST LIMITS · HOST SIDE

Host attack surface that remains

KVM — host kernel
Shared by every KVM product. Patched with the host.

VMM devices — libkrun · Rust
virtio-fs for the root filesystem and bind mounts, block, vsock, console

Host-side parsers — Rust
smoltcp, DNS, TLS termination with rustls, HTTP for authorization and mediation

The VMM process isn't confined — gap
It runs as your user with no seccomp filter, Landlock or namespaces. Firecracker ships a jailer and seccomp filters. A VMM bug means code execution as you, on your machine.

Kubernetes — second layer
The VMM runs in a non-privileged pod that only gets `/dev/kvm` from a device plugin.

> "Design for the assumption that a guest will eventually escape its first boundary."

Cloud Security Alliance, research note on PwnEscape

Next layer to add

Confine the VMM: a seccomp allowlist, Landlock limited to the Agent's own state and mounts, and dropped capabilities. It is cheap compared to a host compromise.

HONEST LIMITS · INSIDE THE BOUNDARY

What the Sandbox doesn’t protect against

Protected

by design

Your machine
the agent has root only inside its own VM

Secret values
never in the VM, only sent to their allowed hosts

Other agents
separate VMs, disks and networks

The host network
only the Platform API is reachable

Not protected

needs other controls

Misuse of a token on its own hosts
anything the PAT allows on github.com, including a public gist

Data leaving without secrets
with `allow: all`, code can be posted to any host

Prompt injection
issues, web pages and dependencies can steer the agent

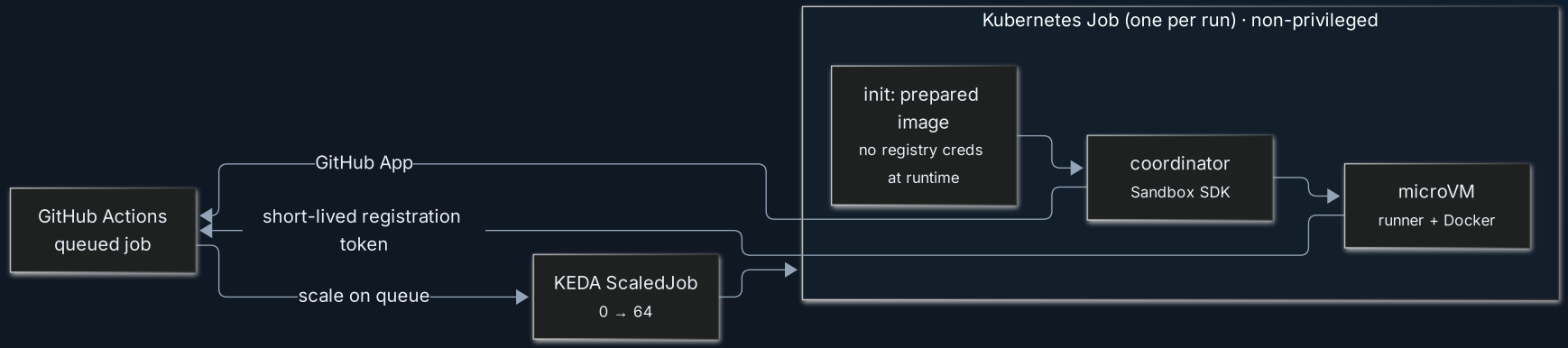
Bad code
review is still the gate, not the Sandbox

Sessions in one Sandbox
same user and tmux server, so one trust boundary

Narrow tokens, tighter `allow` lists and review shrink the gap. The policy engine is where stricter rules can go.

ALTINN-STUDIO · SRC/CI/GITHUB-RUNNER

Same boundary, other workloads



Untrusted PR code

CI runs whatever a pull request contains. One VM per job, with a private root, workspace and Docker data.

Credentials stay out

The GitHub App key stays in the coordinator. The guest gets a one-time token and drops it before the runner starts.

Docker without privilege

Containers run inside the VM instead of in a privileged pod or on a mounted Docker socket.

The Sandbox SDK has no agent concepts. Any workload that runs untrusted code can use the same boundary.

FUTURE · NOT DECIDED

What’s next



Company AI gateway

Send model and MCP traffic through Digdir's gateway: one place for logging, policy and cost. MCP servers are proxied and vetted centrally instead of per agent. Open models for security work on code that must not leave Digdir.



Plugin architecture

New capabilities as host plugins behind a Platform API. They run on the trusted side with their own authorization, so the Sandbox gets a capability without getting the credentials behind it.



Cloud agents

Software factories: agents started from an issue or a pipeline and run on Kubernetes, like the GitHub runners. The same boundary, at a scale where identity, review and tenancy become the hard questions.

THANKS

Questions?

github.com/digdir/digdir-agents · `agentctl/README.md` · `agentctl/CHANGELOG.md`

github.com/digdir/microsandbox · github.com/digdir/libkrunfw

Sources: theregister.com · pwn.ai/blog/pwnescape · linuxcvetracker.com · labs.cloudsecurityalliance.org